

SAMPLE TABLE : Employee (We will use only this table in all examples)

emp_id	name	department	salary	age	city	joining_date	manager_id
1	Aman Views	IT	80000	28	Delhi	2022-01-15	NULL
2	Rahul	HR	60000	30	Mumbai	2021-03-10	1
3	Priya	Finance	75000	26	Bangalore	2022-06-18	1

1. DATABASE & TABLE OPERATIONS

- Create Database
`CREATE DATABASE AmanViewsDB;`
- Use Database
`USE AmanViewsDB;`
- Delete Database
`DROP DATABASE AmanViewsDB;`
- Create Table
`CREATE TABLE Employee (
emp_id INT PRIMARY KEY,
name VARCHAR(50),
department VARCHAR(50),
salary INT,
age INT,
city VARCHAR(50),
joining_date DATE,
manager_id INT
);`
- Alter Table (Add Column)
`ALTER TABLE Employee ADD email VARCHAR(50);`
- Update Table (Modify Column)
`ALTER TABLE Employee
MODIFY COLUMN salary DECIMAL(10,2);`
- Delete Table
`DROP TABLE Employee;`

2. INSERT, UPDATE, DELETE (DML)

- Insert Into Table
`INSERT INTO Employee
(emp_id, name, department, salary, age,
city, joining_date, manager_id)
VALUES (4, 'Aman Views', 'IT', 90000,
29, 'Pune', '2024-01-01', 1);`
- Update Data
`UPDATE Employee
SET salary = 85000
WHERE emp_id = 1;`
- Delete Data
`DELETE FROM Employee
WHERE emp_id = 2;`

3. BASIC QUERIES & FILTERING

- ① SELECT
 - Retrieve all data
`SELECT * FROM Employee;`
- ② WHERE
 - Filter rows
`SELECT * FROM Employee
WHERE salary > 60000;`
- ③ DISTINCT
 - Get unique values
`SELECT DISTINCT department FROM Employee;`
- ④ ORDER BY
 - Sort results
`SELECT * FROM Employee
ORDER BY salary DESC;`
- ⑤ LIMIT / TOP
 - Limit number of rows
`SELECT * FROM Employee
ORDER BY salary DESC LIMIT 2;`
-- SQL Server : TOP 2
`SELECT TOP 2 * FROM Employee
ORDER BY salary DESC;`
- ⑥ LIKE
 - Pattern matching
`SELECT * FROM Employee WHERE name LIKE 'A%';` -- starts with A
`SELECT * FROM Employee WHERE name LIKE '%Views';` -- ends with Views
`SELECT * FROM Employee WHERE name LIKE '_m%';` -- second char m
- ⑦ IN
 - Match any value in list
`SELECT * FROM Employee WHERE department IN ('IT', 'HR');`
- ⑧ BETWEEN
 - Between two values
`SELECT * FROM Employee WHERE salary BETWEEN 60000 AND 80000;`
- ⑨ IS NULL / IS NOT NULL
 - Check for NULL values
`SELECT * FROM Employee WHERE manager_id IS NULL;`
`SELECT * FROM Employee WHERE manager_id IS NOT NULL;`
- ⑩ ALIAS (AS)
 - Give temporary name to columns/table
`SELECT emp_id AS id, name AS employee_name, salary
FROM Employee;`

Key Complete SQL Concepts in Two Pages

1. AGGREGATE FUNCTIONS

- COUNT() - Counts rows
- SUM() - Sum of values
- AVG() - Average of values
- MIN() - Minimum value
- MAX() - Maximum value

```
SELECT department, COUNT(*) AS emp_count,  
       AVG(salary) AS avg_salary  
FROM Employee  
GROUP BY department;
```

2. GROUP BY & HAVING

GROUP BY - Groups rows having same values

```
SELECT department, COUNT(*) AS emp_count  
FROM Employee  
GROUP BY department;
```

HAVING - Filter groups

```
SELECT department, AVG(salary) AS avg_salary  
FROM Employee  
GROUP BY department  
HAVING AVG(salary) > 65000;
```

3. CASE (Conditional Logic)

```
SELECT name, salary,  
CASE  
    WHEN salary >= 80000 THEN 'High'  
    WHEN salary >= 60000 THEN 'Medium'  
    ELSE 'Low'  
END AS salary_level  
FROM Employee;
```

4. JOINS (Combine Data from Multiple Tables)

- INNER JOIN - Returns matching rows



```
SELECT E.emp_id, E.name, D.department  
FROM Employee E  
INNER JOIN Department D  
ON E.department = D.department;
```

- LEFT JOIN - All rows from left table



```
SELECT E.emp_id, E.name, D.department  
FROM Employee E  
LEFT JOIN Department D  
ON E.department = D.department;
```

- RIGHT JOIN - All rows from right table



```
SELECT E.emp_id, E.name, D.department  
FROM Employee E  
RIGHT JOIN Department D  
ON E.department = D.department;
```

5. SET OPERATORS

- UNION - Combines results & removes duplicates

```
SELECT name FROM Employee WHERE city = 'Delhi'  
UNION  
SELECT name FROM Employee WHERE city = 'Mumbai';
```

- UNION ALL - Combines results & keeps duplicates

```
SELECT name FROM Employee WHERE city = 'Delhi'  
UNION ALL  
SELECT name FROM Employee WHERE city = 'Mumbai';
```

6. SUBQUERY

- Subquery returns a single value

```
SELECT name, salary  
FROM Employee  
WHERE salary > (SELECT AVG(salary) FROM Employee);
```

- Subquery returns multiple rows

```
SELECT name, department  
FROM Employee  
WHERE department IN (SELECT department FROM Employee  
                     WHERE salary > 60000);
```

7. WINDOW FUNCTIONS

- ROW_NUMBER() - Unique number for each row
- RANK() - Rank with gaps
- DENSE_RANK() - Rank without gaps

```
SELECT name, salary, department,  
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num,  
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rn,  
       DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS drnk  
FROM Employee;
```

8. COMMON FUNCTIONS

UPPER(str)	- Convert to upper case
LOWER(str)	- Convert to lower case
LENGTH(str)	- Length of string
ROUND(num, n)	- Round number to n decimals
COALESCE(a, b, ...)	- First non-null value
NOW()	- Current date & time
DATE_FORMAT(date, '%Y-%m-%d')	- Format date

9. SQL EXECUTION ORDER

```
FROM → WHERE → GROUP BY → HAVING →  
→ SELECT → ORDER BY → LIMIT / TOP
```

10. QUICK REFERENCE (DCL, TCL)

DCL (Data Control Language)	TCL (Transaction Control Language)
• GRANT - Give access • REVOKE - Remove access	• COMMIT - Save changes • ROLLBACK - Undo changes • SAVEPOINT - Set point

11. NOTES

INTERVIEW REALITY CHECK

90% OF CANDIDATES MAKE THIS ONE MISTAKE.

It's not lack of
knowledge.
It's lack of
REVISION.

30+ HANDWRITTEN
PDF NOTES

THEY KEEP LEARNING...

- ❌ Another YouTube playlist
- ❌ Another course
- ❌ Another DSA sheet
- ❌ Another roadmap



BUT THEY NEVER...

REVISE.



Revise Faster
Save time before
interviews

Remember Longer
Handwritten notes
help you retain

Interview Focused
Covers what really
gets asked

PREPARE YOUR NEXT INTERVIEW.

START REVISING.



CHECK LINK IN BIO